

Code Translation using Transformer Models: Java ↔ C#

Name: Sagri Mohammed Umer **Roll No.:** 24074036 **Program:** 2nd Year, IDD, CSE

Abstract

Code translation is an important task in software engineering and Natural Language Processing (NLP), where the goal is to convert source code from one programming language to another while preserving functionality and structure. This project focuses on Java to C# code conversion using transformer-based models. A baseline CodeBERT model is evaluated first, followed by task-specific fine-tuning on the CodeXGLUE Java-C# dataset. The system is evaluated using BLEU, ROUGE-L, and Exact Match metrics. A Django-based web interface is also developed to demonstrate real-time translation.

1 Introduction

Automatic code translation is useful for software migration, modernization of legacy systems, and cross-platform development. Traditional rule-based converters often fail for complex real-world code because programming languages differ in syntax, library usage, naming conventions, and structural patterns.

Recent transformer-based models have shown strong performance in natural language and source code understanding tasks. CodeBERT is a pre-trained model designed for programming languages and natural language. In this project, CodeBERT is used as the base architecture and fine-tuned for Java ↔ C# conversion. The main focus is on evaluation, experimentation, and practical demonstration.

2 Methodology

2.1 Dataset

The CodeXGLUE Java-C# dataset is used for training and evaluation. It contains aligned Java and C# functions collected from real-world open-source repositories.

Split	Number of Samples
Training	10,295
Validation	499
Testing	1,000

The dataset mainly contains function-level code snippets, where each Java function is aligned with its corresponding C# implementation.

2.2 Model and Pipeline

The baseline uses pre-trained CodeBERT without task-specific fine-tuning. Since CodeBERT is not directly trained for code translation, its performance as a baseline is limited. For the fine-tuned model, CodeBERT is used as the encoder along with a Transformer decoder to generate target C# code.

The pipeline consists of data preprocessing, tokenization, baseline evaluation, fine-tuning, metric-based evaluation, and Django-based web deployment.

2.3 Evaluation Metrics

The model is evaluated using:

- **BLEU Score:** Measures n-gram overlap between generated and reference code.
- **ROUGE-L:** Measures longest common subsequence similarity.
- **Exact Match:** Measures percentage of predictions exactly matching the reference.



3 Results and Discussion

Model	BLEU	ROUGE-L	Exact Match
Baseline CodeBERT	6.11	–	–
Fine-tuned CodeBERT	34.59	0.678	21.84%

The baseline model gives a low BLEU score because it is not specifically trained for Java to C# generation. After fine-tuning, the BLEU score improves significantly to 34.59, showing that the model learns task-specific syntax and structural mappings. The ROUGE-L score of 0.678 indicates strong sequence-level similarity, while Exact Match of 21.84% shows that a noticeable portion of predictions exactly match the target output.

The visual analysis also supports these observations. The function length distribution shows that the dataset mainly contains short and medium-length functions. The length correlation graph indicates that predicted code length generally follows reference length. The similarity heatmap shows both strong matches and challenging cases, especially for longer or more complex functions.

4 Demo System

A Django-based web application was developed to demonstrate real-time code translation. The user can paste Java code into the interface and obtain generated C# output using the fine-tuned checkpoint. This makes the project more practical and shows how the trained model can be integrated into an application pipeline.

5 Conclusion

This project demonstrates that transformer-based models can be adapted for code translation through fine-tuning. The baseline CodeBERT model performs poorly for direct translation, but fine-tuning on the CodeXGLUE Java-C# dataset significantly improves performance. Future improvements may include larger code-specific models, better decoding strategies, syntax-aware evaluation, and semantic correctness checking through compilation or unit tests.

6 References

1. [Lu et al., CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation, 2021.](#)
2. [Feng et al., CodeBERT: A Pre-Trained Model for Programming and Natural Languages, 2020.](#)
3. [HuggingFace Transformers Documentation.](#)
4. [Papineni et al., BLEU: A Method for Automatic Evaluation of Machine Translation, 2002.](#)
5. [Lin, ROUGE: A Package for Automatic Evaluation of Summaries, 2004.](#)